

## Übung 2 – Braccio-Mehrgelenk-Roboter

In dieser Übung wird ein Braccio-Mehrgelenkroboter mit 6 Freiheitsgraden betrachtet. Die Übung dient als Vorbereitung des zugehörigen Labors.

**Die Vorbereitung der nachfolgenden Übungsaufgaben ist Voraussetzung für eine erfolgreiche Teilnahme am Labor!**

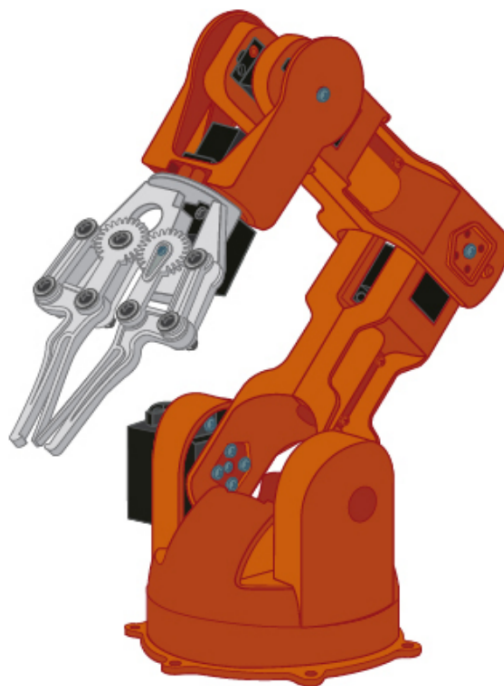


Abbildung 1. Der in dieser Übung betrachtete Braccio-Roboter.

Zur simulationsgestützten Inbetriebnahme stehen die nachfolgenden Matlab/Simulink-Dateien zur Verfügung:

| Datei                                     | Hinweise                                               |
|-------------------------------------------|--------------------------------------------------------|
| <a href="#">Braccio_kinematics.m</a>      | m-Skript zur Initialisierung des Simulationsmodells    |
| <a href="#">Braccio_CtrlSimMdl.slx</a>    | Simulink-Modell zur Simulation des Roboters            |
| <a href="#">Braccio_iksessiondata.mat</a> | Inverse-Kinematics-Designer Session                    |
| <a href="#">Braccio_plotConfig.m</a>      | m-Skript zur Anzeige von Posen (Configurations)        |
| <a href="#">Braccio_plotSimResults.m</a>  | m-Skript zur Visualisierung von Simulationsergebnissen |
| <a href="#">Braccio_lib.slx</a>           | Simulink-Bibliothek mit den Subsystemen des Roboters   |
| <a href="#">Braccio_HMI.png</a>           | Schematische Darstellung HMI-Bedienkonzept             |

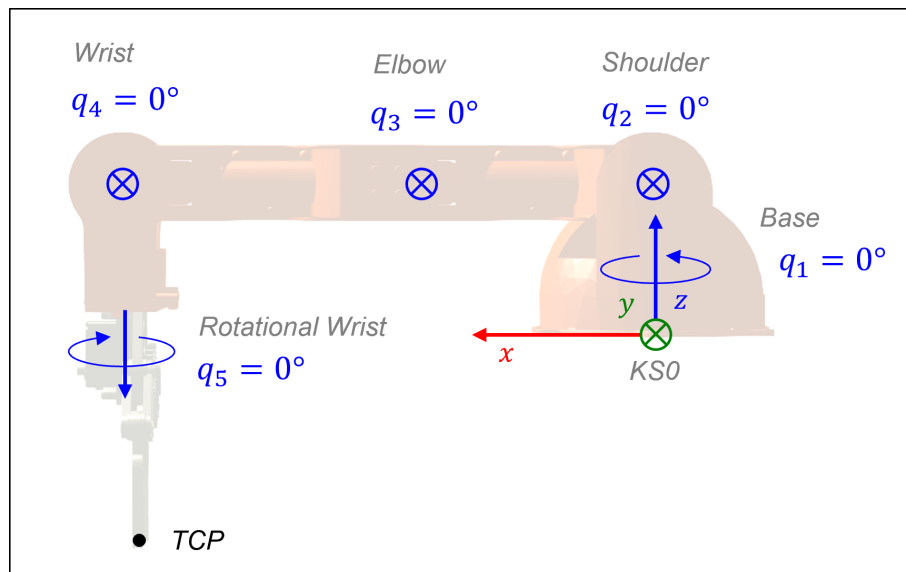


Abbildung 2. Kinematik des Braccio-Roboters mit seinen 5 rotatorischen Freiheitsgraden. Der 6. Freiheitsgrad ist die (nicht dargestellte) Stellung des Greifers. Eingezeichnet ist der Tool-Center-Point (TCP) am Greifer des Roboters.

Die Kinematik des Roboters wird anhand der nachfolgenden DH-Parameter beschrieben:

| Element $i$   | $d_i$ [mm] | $a_i$ [mm] | $\alpha_i$ [deg] | $\theta_i$ [deg] | $q_{i,\min}$ [deg] | $q_{i,\max}$ [deg] |
|---------------|------------|------------|------------------|------------------|--------------------|--------------------|
| Base (1)      | 71,5       | 0          | 90               | 0                | -90                | 90                 |
| Shoulder (2)  | 0          | 125        | 0                | 90               | 15                 | 165                |
| Elbow (3)     | 0          | 125        | 0                | 0                | -90                | 90                 |
| Wrist (4)     | 0          | 0          | 90               | 90               | 0                  | 180                |
| Wrist Rot (5) | 130        | 0          | 0                | 0                | -90                | 90                 |
| Gripper (6)   | –          | –          | –                | –                | 10                 | 73                 |

- a) Parametrieren Sie die DH-Parameter im Initialisierungsskript `Braccio_kinematics.m` an den mit „TODO“ gekennzeichneten Stellen.

Auf Basis der DH-Parameter wird im Skript `Braccio_kinematics.m` ein Rigid-Body-Tree-Objekt zur Beschreibung der Roboter-Kinematik erzeugt.

- b) Überprüfen Sie das Rigid-Body-Tree-Objekt `braccioRobot` mit Hilfe des `show`-Befehls (ca. Zeile 188). Nutzen Sie die abschnittsweise Ausführung des Skripts (Strg+Enter).

Zum Schutz der Umgebung soll der Arbeitsraum des Roboters durch eine „virtuelle Wand“ eingeschränkt werden. Dazu soll eine virtuelle Wand an der Position  $x = -200$  mm positioniert werden.

- c) Machen Sie sich mit den Befehlen `collisionBox` und `trvec2tform` vertraut. Parametrieren Sie die virtuelle Wand gemäß der Vorgabe an der mit „TODO“ gekennzeichneten Stelle.

Zur Vorgabe von kollisionsfreien Roboter-Posen (Configurations) kann in Matlab der Inverse Kinematics Designer genutzt werden.

- d) Starten Sie den Inverse Kinematics Designer durch Ausführung des entsprechenden Abschnitts im Skript und machen Sie sich mit seiner Funktionsweise vertraut.
- (1) Ergänzen Sie die zuvor parametrisierte virtuelle Wand mit Hilfe der Schaltfläche „Add Collision Object.“ Überprüfen Sie die Position der visualisierten Wand.
  - (2) Verändern Sie die Roboter-Pose durch Anwählen der Gelenke und Vorgabe des jeweiligen Gelenkwinkels über den Schieberegler.
  - (3) Speichern Sie die Posen durch einen Klick auf „Store Configuration“.
  - (4) Überprüfen Sie die Abfolge mit Hilfe der Pfeil-Buttons.
  - (5) Überprüfen Sie, ob die gewählten Posen kollisionsfrei sind. Nutzen Sie dazu die Funktion „Check Collisions“.
  - (6) Exportieren Sie die Abfolge mit den gewählten Configurations..

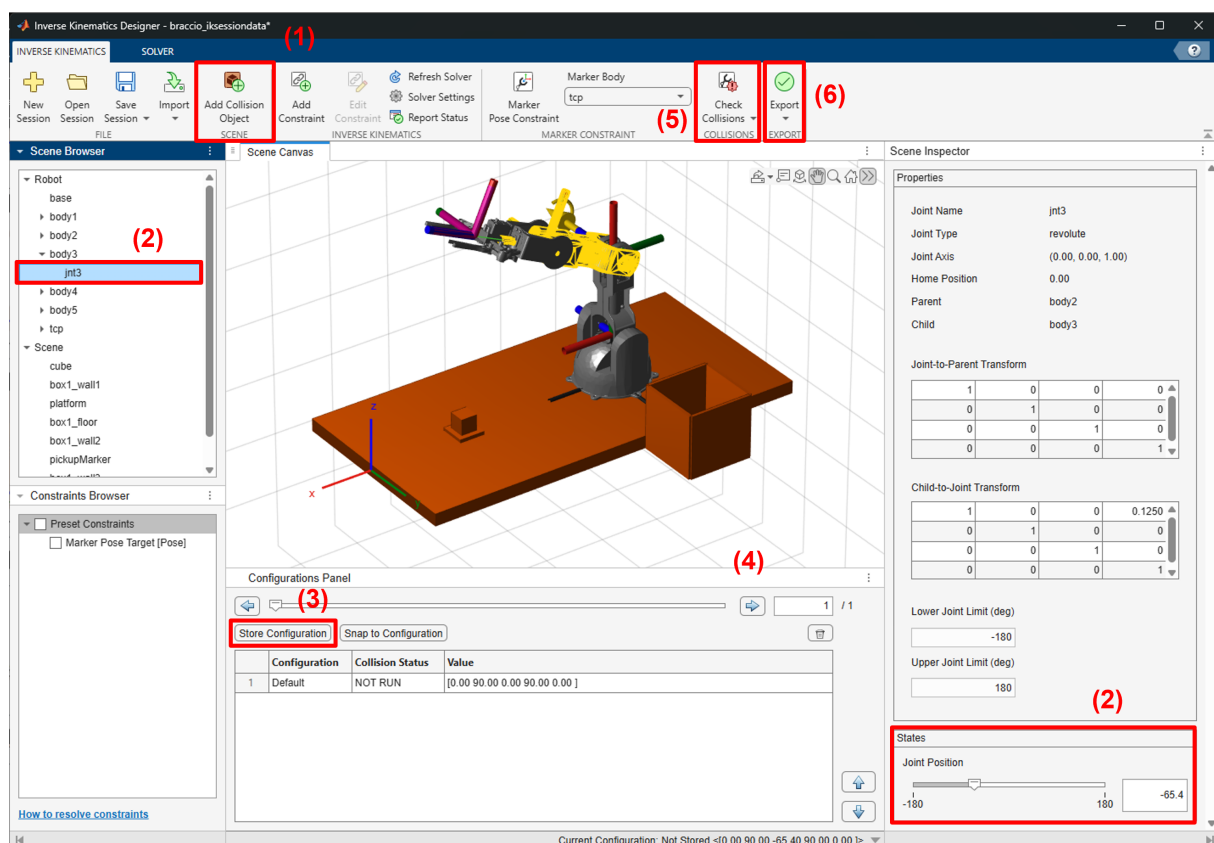


Abbildung 3. Der Inverse-Kinematics-Designer zur Vorgabe und Überprüfung von Roboter-Posen.

Mit Hilfe eines Simulink-Simulationsmodells sollen die entwickelten Funktionen zur Roboter-Steuerung getestet werden. Die Visualisierung erfolgt dabei mit Hilfe der Unreal-Engine.

- e) Öffnen Sie das Simulationsmodell `Braccio_CtrlSimMdl.slx` und machen Sie sich mit dessen Aufbau vertraut.
- Starten Sie das Simulationsmodell im manuellen Modus. Setzen Sie dazu mit Hilfe des Radio-Buttons „CtrlMode = 1“.
  - Zum Start der Bewegung ist nach jedem Modus-Wechsel ein Klick auf den Button „RUN/PAUSE“ erforderlich, siehe Abb. 5.
  - Geben Sie über die Slider verschiedene Gelenkstellungen vor. Beachten Sie, dass die Vorgabe über Servo-Winkel  $\varphi$  erfolgt, welche gegenüber den Gelenkwinkeln  $q$  die in Abb. 4 dargestellte Verschiebung aufweisen, so dass sich für  $\varphi_1 = \varphi_2 = \varphi_3 = \varphi_4 = \varphi_5 = 0$  eine aufrechte „Default“ Pose ergibt.
  - Beobachten Sie das resultierende Verhalten mit Hilfe der Animation.

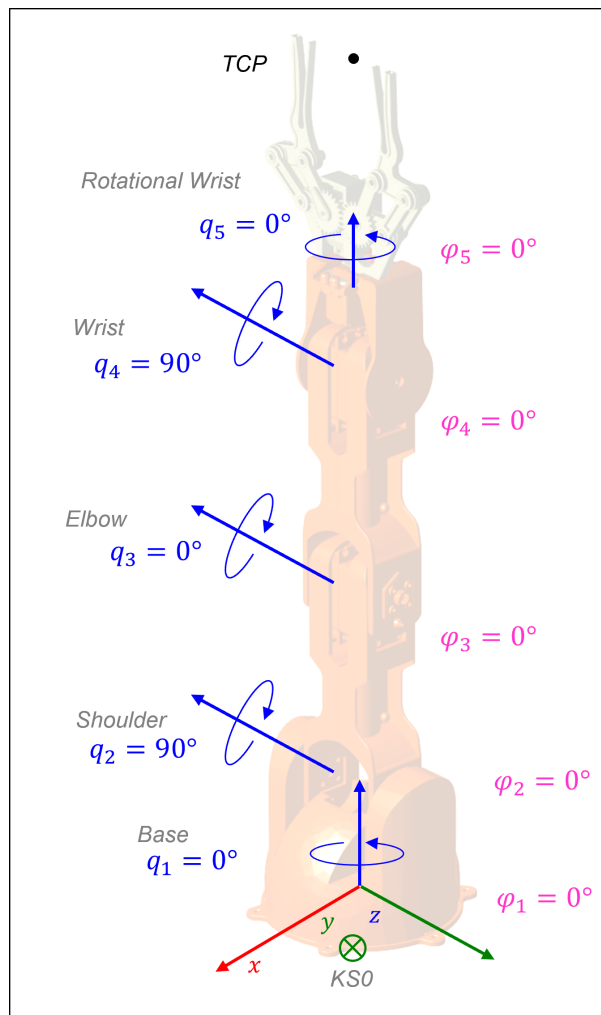


Abbildung 4. Default-Pose des Roboters. Dargestellt sind die zugehörigen Gelenkwinkel  $q$  sowie die Servo-Winkel zur Ansteuerung.

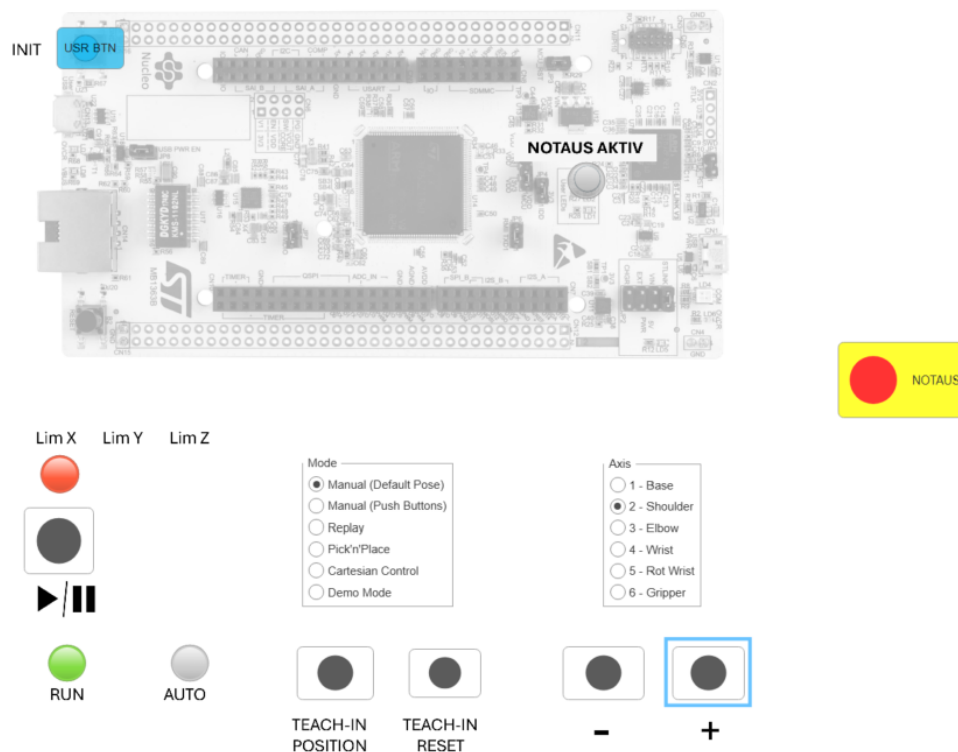


Abbildung 5. Bedienpanel für den Roboter. Nach einem Modus-Wechsel ist zum Start der Bewegung ein Klick auf den Button „RUN/PAUSE“ erforderlich.

- f) Die Position des TCPs soll ausgegeben und überwacht werden. Nutzen sie dazu den vorbereiteten Matlab-Function-Block **ForwardKinematics**.
- Implementieren Sie die homogenen DH-Transformationsmatrizen  $T_1, T_2, T_3, T_4, T_5$  unter Ausnutzung der als Eingang angelegten Gelenkwinkel.
  - Berechnen Sie Position und Orientierung des TCPs anhand der DH-Matrizen.
  - Berechnen Sie die Position der Punkte B (Elbow-Joint) und C (Wrist-Joint).
  - Machen Sie sich mit dem Bereich „Arbeitsraumüberwachung“ vertraut.
  - Überprüfen Sie Ihre Lösung mit Hilfe der Animation. Bei Verletzung der Arbeitsraumbeschränkung sollte die virtuelle Wand rot aufleuchten und die entsprechende Dashboard-Leuchte im Simulink Modell angehen.
- g) Zur Realisierung einer Pick-and-Place-Aufgabe sollen die zuvor mit Hilfe des Inverse Kinematics Designers bestimmten Posen nacheinander angefahren werden. Nutzen Sie dazu das Subsystem **PickAndPlace**.
- Machen Sie sich mit dem Block **Polynomial Trajectory** vertraut.
  - Übernehmen Sie die zuvor exportierten Waypoints als Eingang.
  - Geben Sie passende TimePoints vor.
  - Wählen Sie als **CtrlMode** = 4 (Pick-and-Place)
  - Überprüfen Sie das resultierende Verhalten mit Hilfe der Visualisierung.

- h) Alternativ zur Vorgabe der zuvor ermittelten Waypoints steht auch eine Teach-In-Funktion zur Verfügung.
- Wählen Sie als **CtrlMode** = 2 (Manual Push Buttons).
  - Fahren Sie mittels der +/- Buttons die gewünschten Positionen an.
  - Speichern Sie die gewünschten Posen mit dem Button „TEACH-IN POSITION“.
  - Testen Sie das Abfahren der eingestellten Posen durch Wechsel in den „Replay“-Betriebsmodus (**CtrlMode** = 3).
- i) Abschließend sollen Bauteile vom Roboter in ein Regal einsortiert werden. Dazu sollen mit Hilfe des Inverse Kinematics Designers 8 Posen definiert werden. Ein Verfahren des TCPs in kartesischen x-z-Koordinaten soll dann mit Hilfe des **CartesianPath**-Subsystems realisiert werden.
- Ermitteln Sie mit Hilfe des Inverse Kinematics Designers die erforderlichen Posen und exportieren Sie diese in den Workspace.
  - Übertragen Sie die ermittelten Werte für die x- und z-Positionen im Ini-Skript an den mit „TODO“ gekennzeichneten Stellen.
  - Übertragen Sie die initiale Konfiguration der Gelenkwinkel.
  - Führen Sie das Modell aus mit **CtrlMode** = 5 (Cartesian Path).
  - Überprüfen Sie das resultierende Verhalten mit Hilfe der Visualisierung.